UNIVERSITY INSTITUTE OF COMPUTING

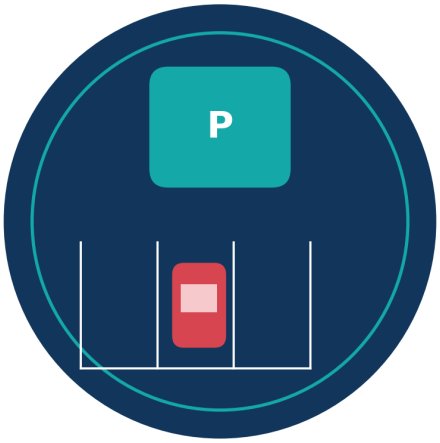
Bachelor of Computer Applications (BCA)

PROJECT BASED LEARNING (PBL) / CASE STUDY REPORT

on

SMART PARKING MANAGEMENT SYSTEM

An Educational Software Engineering Case Study



Subject: Software Engineering

Name	Prajwal
UID	25BCD10271
Course	BCA
Section	25BCD4-B
Semester	3rd
Submitted To	Jitendra Kumar
Academic Session	2026–2027

University Institute of Computing

CERTIFICATE

This is to certify that the PBL / Case Study report entitled “Smart Parking Management System” is a bonafide work carried out by **Prajwal**, UID 25BCD10271, a student of BCA, Section 25BCD4-B, 3rd Semester, University Institute of Computing, in partial fulfilment of the requirements of the subject Software Engineering during the Academic Session 2026–2027.

The work has been completed under my guidance and, to the best of my knowledge, has not been submitted elsewhere for any other purpose. This report is an educational software engineering case study.

_____

Jitendra Kumar
Subject Teacher / Project Guide

Date: _____ Place: _____

DECLARATION

I, **Prajwal**, UID 25BCD10271, a student of BCA, Section 25BCD4-B, 3rd Semester, University Institute of Computing, hereby declare that this PBL / Case Study report entitled “Smart Parking Management System” is my own work, prepared as part of the subject Software Engineering during the Academic Session 2026–2027.

The information in this report has been collected from textbooks and reliable sources listed in the References section. I have not copied any report or project of another student. I further declare that this is an educational case study; it does not contain real user data, real payment details or instructions for building a real parking system.

Date: _____

Place: _____

Signature of Student
Prajwal (UID: 25BCD10271)

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my subject teacher and guide, Mr. Jitendra Kumar, for his valuable guidance, encouragement and support throughout this case study. His explanations of Software Engineering concepts helped me understand how real systems are planned, designed and tested.

I am thankful to the University Institute of Computing for giving me the opportunity to work on a Project Based Learning (PBL) topic. I also thank all faculty members for creating a good learning environment.

Finally, I thank my family and friends for their constant support and motivation during the preparation of this report.

Prajwal
BCA, Section 25BCD4-B, 3rd Semester

ABSTRACT

A smart parking management system is a software-controlled system that tracks the state of every parking slot, guides drivers to a free slot, controls the entry and exit gates, calculates the parking fee and records every event. Because it controls gates and handles personal data and payments, its software must be dependable, secure and easy to use.

The report explains the problem statement, objectives and scope, compares the existing manual approach with a proposed software-controlled design, and describes the main system components, software requirements and a three-layer architecture. The design is shown with a flowchart, a UML use case diagram, Data Flow Diagrams, an ER diagram and UML class, sequence and state diagrams. The Software Development Life Cycle (SDLC), an implementation overview, security and risk management, sample test cases and ethical considerations are also discussed, followed by advantages, limitations and future scope.

This work is purely educational. It does not provide a real product, real user data or real payment handling. Its aim is to help a BCA student understand how requirements, design, software testing and professional responsibility come together when building dependable software.

Keywords: *smart parking, slot management, software engineering, SDLC, UML, DFD, ER diagram, software testing*.

TABLE OF CONTENTS

CERTIFICATE ii
DECLARATION iii
ACKNOWLEDGMENT iv
ABSTRACT v
TABLE OF CONTENTS vi
LIST OF FIGURES vii
LIST OF TABLES viii
ABBREVIATIONS AND GLOSSARY ix
Chapter 1: Introduction 1
Chapter 2: Problem Statement 2
Chapter 3: Objectives 3
Chapter 4: Scope of the Project 3
Chapter 5: Existing System 3
Chapter 6: Proposed System 4
Chapter 7: System Components 5
Chapter 8: Software Requirements 6
Chapter 9: System Architecture 6
Chapter 10: Working of the System 7
Chapter 11: System Flowchart 8
Chapter 12: UML Use Case Diagram 9
Chapter 13: Data Flow Diagram (DFD) 10
Chapter 14: Database Design 11
Chapter 15: UML Design Models 11
Chapter 16: Software Development Life Cycle (SDLC) 14
Chapter 17: Implementation Overview 14
Chapter 18: Security and Risk Management 15
Chapter 19: Software Testing 16
Chapter 20: Ethical and Professional Considerations 17
Chapter 21: Advantages 17
Chapter 22: Limitations 18
Chapter 23: Future Scope 18
Chapter 24: Conclusion 18
References 19
Appendix A: Requirements Traceability Matrix 19

LIST OF FIGURES

Figure 1.1: Simplified schematic of a smart parking lot 1
Figure 6.1: Conceptual block diagram of the proposed system 4
Figure 7.1: Main components of the smart parking system 5
Figure 9.1: Three-layer architecture of the smart parking system 7
Figure 11.1: Flowchart of vehicle entry, parking and exit 8
Figure 12.1: UML use case diagram 9
Figure 13.1: Level 0 DFD 10
Figure 13.2: Level 1 DFD 10
Figure 14.1: Entity-Relationship (ER) diagram 11
Figure 15.1: UML class diagram of the main classes 12
Figure 15.2: UML sequence diagram for a vehicle entry and exit 13
Figure 15.3: UML state diagram of a parking slot 13
Figure 16.1: SDLC phases 14

LIST OF TABLES

Table 2.1: Main challenges 2
Table 6.1: Existing versus proposed system 5
Table 7.1: Components and their purpose 5
Table 8.1: Functional requirements 6
Table 8.2: Non-functional requirements 6
Table 9.1: Software modules and responsibilities 7
Table 10.1: Examples of system behaviour 8
Table 12.1: Actors and use cases 9
Table 12.2: Use case description: Book a Slot 9
Table 14.1: Table structure 11
Table 16.1: SDLC phases and activities 14
Table 18.1: Security measures 16
Table 18.2: Sample risk table 16
Table 19.1: Sample test cases 16
Table A.1: Requirement traceability 19

ABBREVIATIONS AND GLOSSARY

Abbreviation	Full Form
BCA	Bachelor of Computer Applications
PBL	Project Based Learning
SPMS	Smart Parking Management System
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
DFD	Data Flow Diagram
ER	Entity-Relationship
FR / NFR	Functional / Non-Functional Requirement
UI	User Interface
QR	Quick Response (code)
API	Application Programming Interface
TLS	Transport Layer Security
UPI	Unified Payments Interface
PK / FK	Primary Key / Foreign Key
IEEE	Institute of Electrical and Electronics Engineers
ACM	Association for Computing Machinery

Term	Meaning
Slot	One marked parking space for a single vehicle.
Sensor	A device that detects whether a slot is occupied.
Barrier	The gate arm that opens for a valid vehicle.
Ticket	A record (QR or plate based) that links a vehicle to a slot and entry time.
Booking	A reservation of a slot for a time period.
Actor	A user or external system that interacts with the system.
Data store	A place where data is kept, such as a table.

Chapter 1: Introduction

1.1 Background

Finding a free parking space in a busy city is a daily problem. Drivers drive around lots and basements looking for an empty slot, which wastes fuel and time and adds to traffic and pollution. Parking owners, on the other hand, often rely on paper tickets, manual cash handling and guards who watch the gates, which leads to errors, queues and revenue leakage. A smart parking management system uses software, simple sensors and a database to solve these problems. This case study uses such a system to learn how a real-time information system is designed.

1.2 Basic Concepts

- Slot: one marked parking space; its state can be Free, Reserved, Occupied or Out of Service.
- Sensor: a device that tells the software whether a slot is occupied.
- Gate and barrier: the entry and exit points that open only for a valid vehicle.
- Ticket: a QR or plate-based record that links a vehicle, a slot and the entry time.
- Booking: a reservation of a slot made in advance by a driver.
- Fee: the amount calculated from the time a vehicle stays parked.

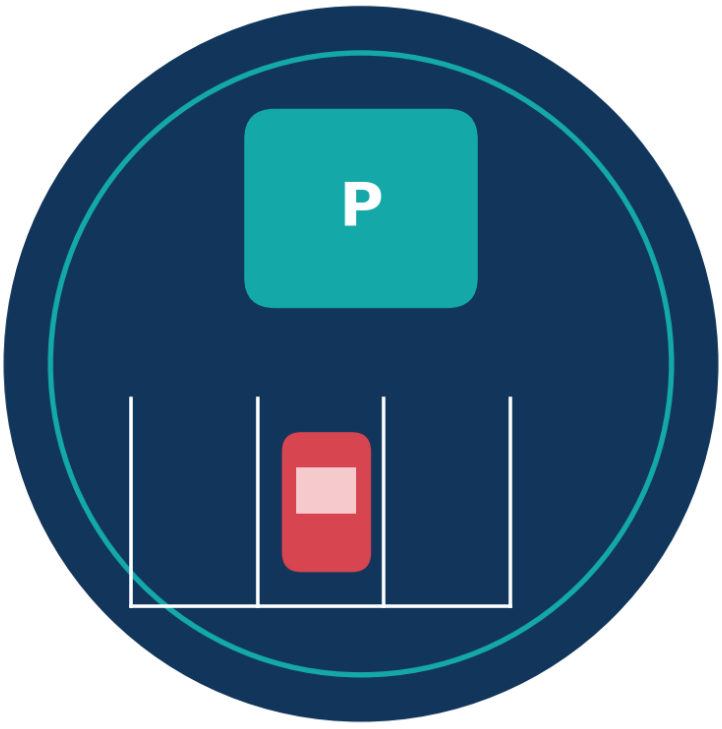


Figure 1.1: Simplified schematic of a smart parking lot

1.3 Why is it a Good Software Engineering Case Study?

A parking system looks simple but contains many important ideas: several types of users, real-time sensor data, gate control, a database, payments, security and error handling. These make it a good example for learning requirements, design models and the SDLC.

1.4 About this Case Study

This report applies Software Engineering concepts to a smart parking management system. The architecture and models shown here are the student's own simplified, conceptual design; they do not describe any commercial product. It is an independent educational study and uses no real user or vehicle data.

1.5 Organisation of the Report

Chapters 2 to 4 define the problem, objectives and scope. Chapters 5 to 8 describe the existing and proposed systems, components and requirements. Chapters 9 to 15 present the design models. Chapters 16 to 20 cover SDLC, implementation, security, testing and ethics. Chapters 21 to 24 give advantages, limitations, future scope and the conclusion.

Chapter 2: Problem Statement

Cities and campuses have more vehicles every year, but the number of parking spaces grows slowly. Most parking lots are still managed manually, so the system cannot tell drivers where a free slot is or how long a vehicle has stayed.

2.1 Problems with Manual Parking

- Drivers search for a free slot for a long time, which creates queues at the gate.
- Slots are not tracked, so the lot may look full while some slots are empty.
- Paper tickets can be lost, copied or misused.
- Cash handling causes mistakes and revenue leakage.
- There is no record of who parked where and for how long.
- Pre-booking a slot is not possible.

2.2 Problem Definition

The problem is to design software for a parking lot that tracks the state of every slot, lets drivers register and book slots, controls the entry and exit gates, calculates the parking fee from the parked time, accepts digital payment and gives the operator live reports, while keeping user data and payment records safe.

2.3 Key Challenges for Software Engineers

Table 2.1: Main challenges

Challenge	Meaning
Real-time accuracy	Slot status must match the real lot within seconds.
Reliability	Gates and billing must keep working when a sensor fails.
Security	User data and payments must be protected.
Fairness	Two drivers must never get the same slot.
Usability	Screens must be simple for drivers and staff.

Chapter 3: Objectives

The main objectives of this case study are:

- To understand how a smart parking management system works.
- To identify the functional and non-functional requirements of the system.
- To design the system using an architecture diagram, flowchart, UML diagrams, DFD and ER diagram.
- To apply SDLC phases and basic software testing ideas.
- To study security, privacy and risk management in parking software.
- To understand ethical and professional responsibility in software work.
- To present the work in a clear report and presentation.

Chapter 4: Scope of the Project

Scope defines what this case study covers and what it does not cover.

4.1 Included

- Requirements analysis (functional and non-functional).
- Software architecture and module design.
- Flowchart, UML diagrams, Data Flow Diagrams and ER diagram.
- SDLC explanation, pseudocode and sample test cases.
- Security, privacy and risk discussion.

4.2 Excluded

- Building the real hardware, sensors or barrier gates.
- The internal source code of any commercial parking product.
- Real user data, real vehicle data or real payments.
- Detailed camera and number-plate recognition algorithms.
- Legal and municipal parking regulations.

4.3 Assumptions

- The parking lot has one entry gate and one exit gate.
- Each slot has a sensor that reports free or occupied.
- Every user, vehicle, slot and booking has a unique ID.
- The system has a stable network connection and power backup.

Chapter 5: Existing System

In most lots today, a guard at the gate gives a paper ticket and notes the time. At exit the guard calculates the fee by hand and collects cash. Some lots use a simple ticket machine, but it does not know which slots are free.

5.1 Limitations of the Existing System

- No live information about free slots.

- Long queues at entry and exit during peak hours.
- Tickets can be lost or forged.
- Manual fee calculation causes errors and disputes.
- Records are on paper, so reports are slow and unreliable.

5.2 Software Engineering View

From a software point of view, the existing approach has almost no software, so there is nothing to validate, monitor or log. This chapter does not compare business results; it only shows why a software-controlled design is a useful case study.

Chapter 6: Proposed System

The proposed system is a software-controlled parking platform. Drivers register and log in, search for free slots, book a slot and pay online. Sensors report the state of each slot to the server. The software allocates a slot, controls the gates, calculates the fee and stores every event in a central database.

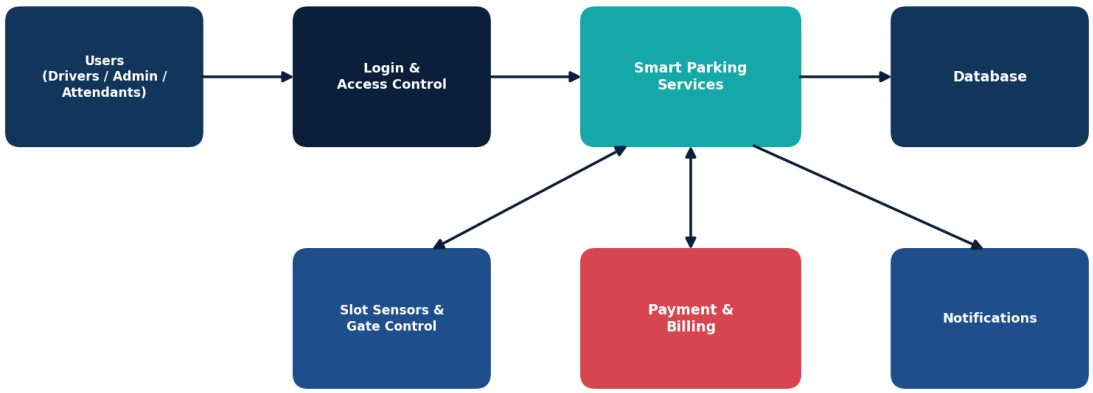


Figure 6.1: Conceptual block diagram of the proposed system

6.1 Key Features

- Registration and login with secure passwords.
- Live view of free and occupied slots.
- Slot booking and cancellation.
- Automatic ticket (QR or plate based) at entry.
- Automatic fee calculation and digital payment.
- Alerts for faults, overstay and full lot.
- Reports for the operator.

Table 6.1: Existing versus proposed system

Point	Existing System	Proposed System
Slot information	None	Live status
Ticketing	Paper ticket	QR / plate based
Fee calculation	Manual	Automatic
Payment	Cash only	Cash and digital
Records	Paper	Database and reports

Chapter 7: System Components

The system has several hardware components and one central software component that controls and monitors all of them.

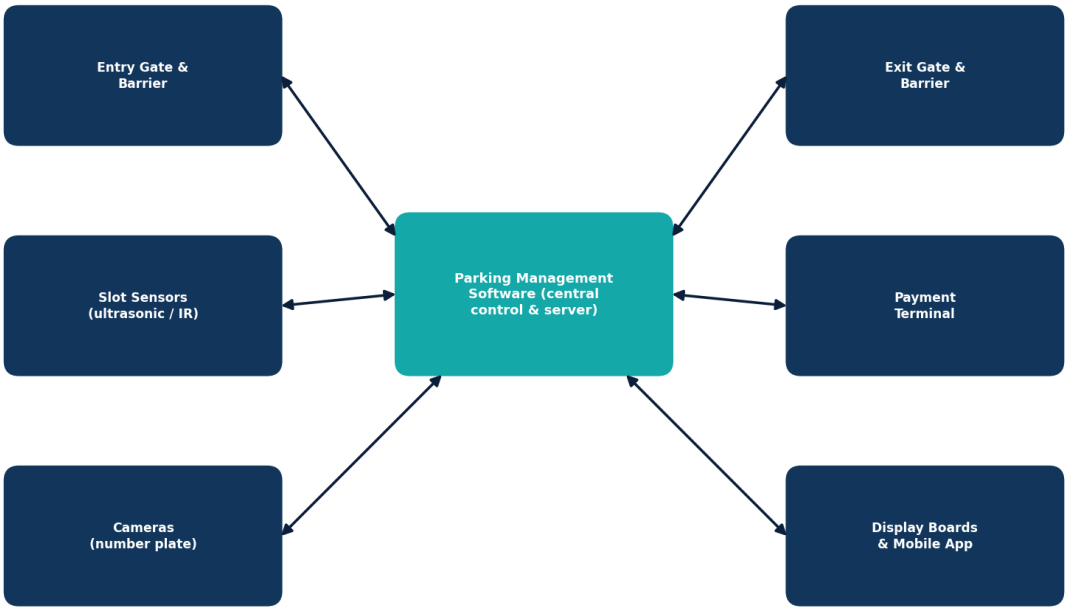


Figure 7.1: Main components of the smart parking system

Table 7.1: Components and their purpose

Component	Purpose
Slot sensors	Detect whether a slot is free or occupied.
Entry / exit gate	Barrier that opens only when the software allows it.
Camera	Reads the number plate or QR code at the gates.
Display boards / app	Show free slots and messages to drivers.
Payment terminal	Accepts card, UPI or wallet payment.
Server and database	Run the software and store all records.

Chapter 8: Software Requirements

Requirements describe what the system must do (functional) and how well it must do it (non-functional).

Table 8.1: Functional requirements

ID	Requirement
FR1	The system shall allow users to register and log in.
FR2	The system shall show the live status of all slots.
FR3	The system shall allow a user to book and cancel a slot.
FR4	The system shall allocate a free slot and issue a ticket at entry.
FR5	The system shall open the gate only for a valid ticket or booking.
FR6	The system shall calculate the fee from the parked time.
FR7	The system shall accept payment and issue a receipt.
FR8	The system shall let the admin manage slots, rates and reports.

Table 8.2: Non-functional requirements

Quality	Description
Performance	Slot status shall update within a few seconds.
Reliability	The gates shall work in a safe manual mode if the server is down.
Security	Passwords are hashed; payment data is sent over secure connections.
Availability	The service shall be available all day.
Usability	Screens shall be simple for drivers and staff.
Maintainability	The design shall be easy to update and fix.

8.3 Constraints

- Sensors and gates have limited accuracy and can fail.
- The system depends on power and network connection.
- Payment must follow the rules of the payment provider.

Chapter 9: System Architecture

A three-layer architecture is used. The presentation layer talks to the application service layer, which in turn uses the data and device layer. Each layer only talks to the layer next to it, which keeps the design simple and easier to test.

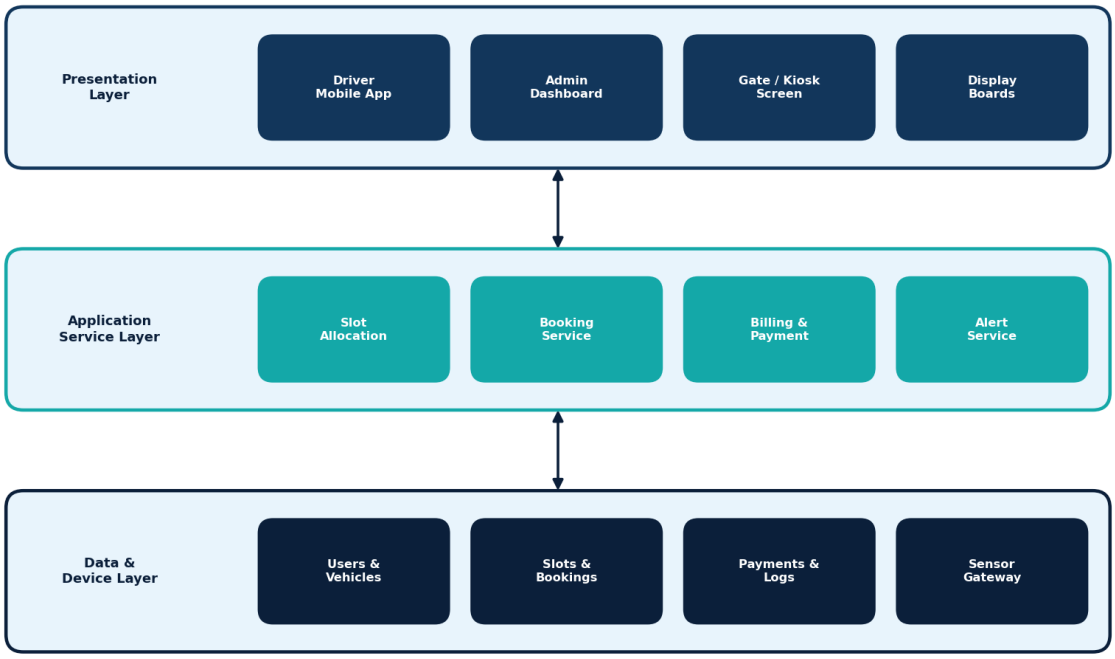


Figure 9.1: Three-layer architecture of the smart parking system

Table 9.1: Software modules and responsibilities

Module	Responsibility
Slot Allocation	Finds a free slot and marks it reserved or occupied.
Booking Service	Creates, changes and cancels bookings.
Billing and Payment	Calculates the fee, takes payment, issues a receipt.
Alert Service	Sends alerts for faults, overstay and a full lot.
Gate Controller	Opens and closes the entry and exit barrier.
Sensor Gateway	Reads slot sensors and reports their state.
Database Tables	Store users, vehicles, slots, bookings and payments.

Chapter 10: Working of the System

The workflow of the system is explained step by step below.

1. The driver registers or logs in; the system checks the details.
2. The driver views free slots and books one, or simply arrives at the gate.
3. At the entry gate the camera reads the plate or QR code.
4. The system checks for a free slot or valid booking.
5. A slot is allocated, a ticket is issued and the barrier opens.
6. The sensor marks the slot Occupied when the vehicle parks.
7. At exit, the ticket or plate is scanned and the fee is calculated.
8. After payment the exit barrier opens, the slot becomes Free and the event is logged.

10.1 Normal and Exceptional Situations

Table 10.1: Examples of system behaviour

Situation	System Response
Normal entry	Allocate slot, issue ticket, open barrier.
Lot full	Show "Parking Full" and keep the barrier closed.
Lost ticket	Find the vehicle by plate; charge the standard fee.
Sensor fault	Mark slot Out of Service and alert the operator.
Payment failed	Keep exit closed; allow retry or call the attendant.
Overstay of booking	Release the slot after the grace time.

Chapter 11: System Flowchart

The flowchart shows the main flow of a vehicle from arrival to exit. The oval is the start or end, rectangles are process steps, diamonds are decisions, and the red boxes are the exception paths.

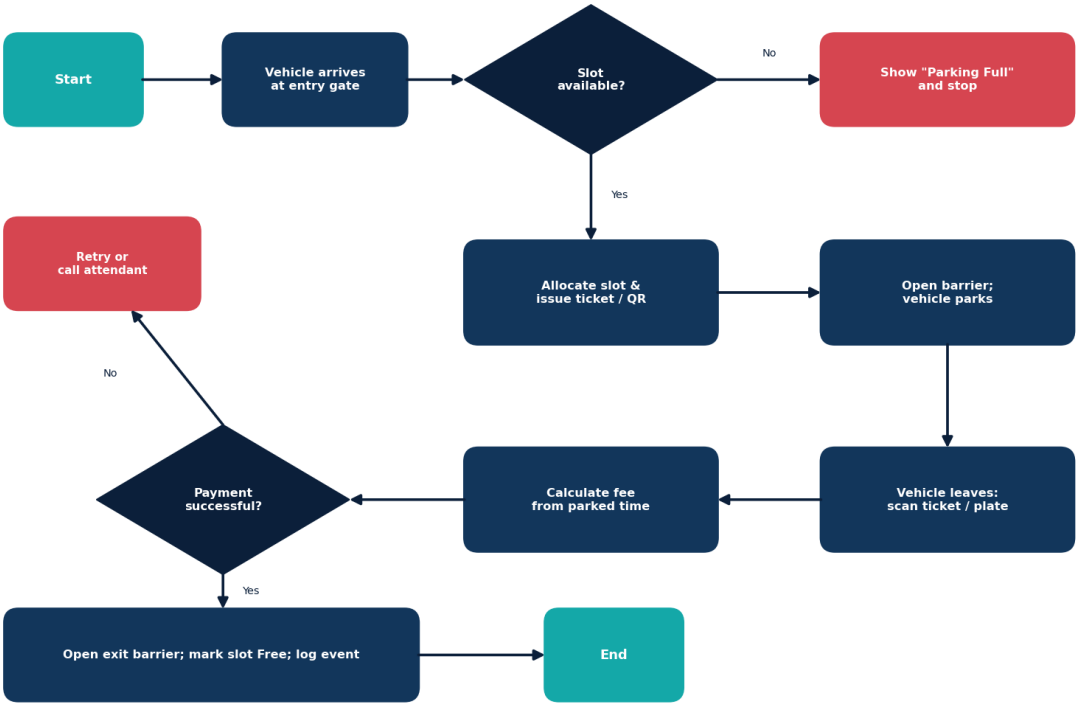


Figure 11.1: Flowchart of vehicle entry, parking and exit

Explanation: When a vehicle arrives, the system checks whether a slot is available. If not, it shows that the lot is full. Otherwise a slot is allocated and a ticket is issued, the barrier opens and the vehicle parks. At exit the fee is calculated and, once payment is successful, the exit barrier opens and the slot is freed.

Chapter 12: UML Use Case Diagram

A use case diagram shows the actors (users or external systems) and how they interact with the system.

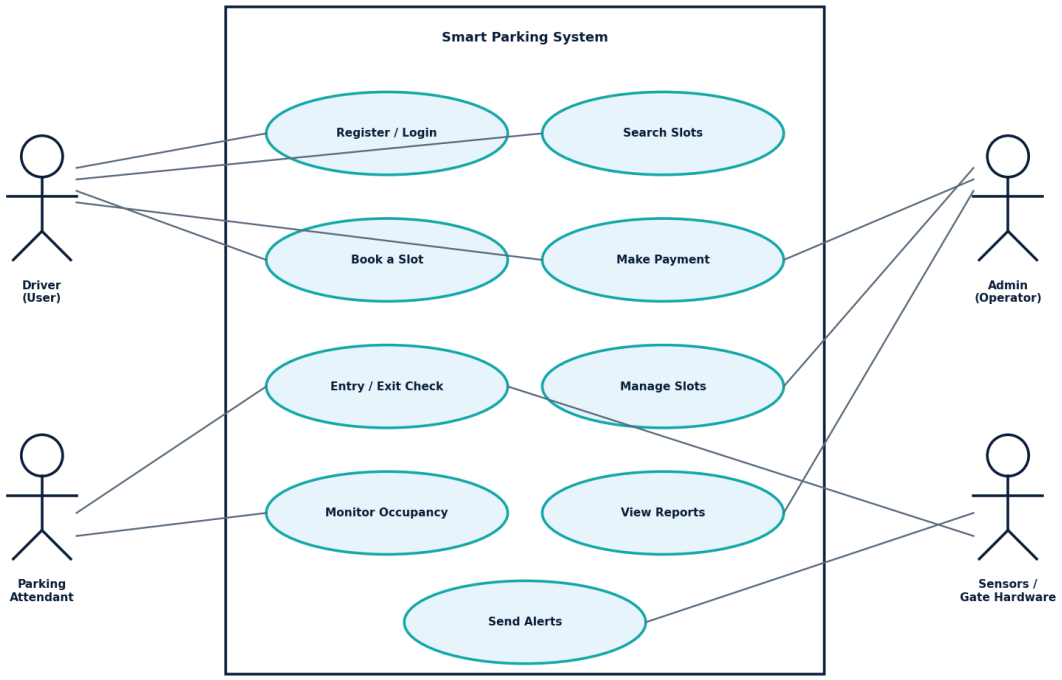


Figure 12.1: UML use case diagram

Table 12.1: Actors and use cases

Actor	Use Cases
Driver (User)	Register / login, search slots, book a slot, make payment.
Parking Attendant	Handle cash payment, help with entry and exit checks.
Admin (Operator)	Manage slots and rates, monitor occupancy, view reports.
Sensors / Gate Hardware	Update slot status and raise fault alerts.

Table 12.2: Use case description: Book a Slot

Item	Description
Actor	Driver (User)
Precondition	The driver is logged in and at least one slot is free.
Main flow	1. The driver selects the zone and time. 2. The system shows free slots. 3. The driver picks a slot. 4. The system reserves it and shows a booking ID.
Alternate flow	If no slot is free, the system shows a message and offers another zone.
Postcondition	The slot is Reserved and the booking is saved.

Chapter 13: Data Flow Diagram (DFD)

A DFD shows how information moves through a system. Rectangles are external entities, circles are processes, open boxes are data stores, and arrows show the data flows.

13.1 Level 0 (Context Diagram)

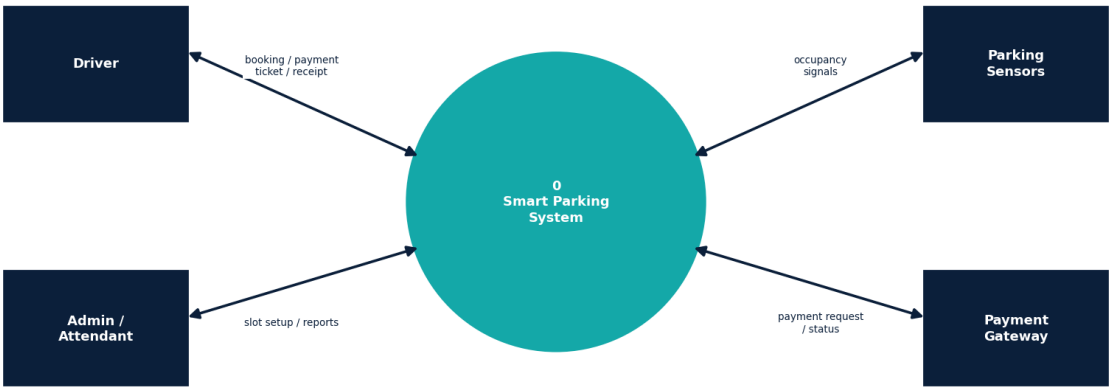


Figure 13.1: Level 0 DFD

13.2 Level 1 DFD

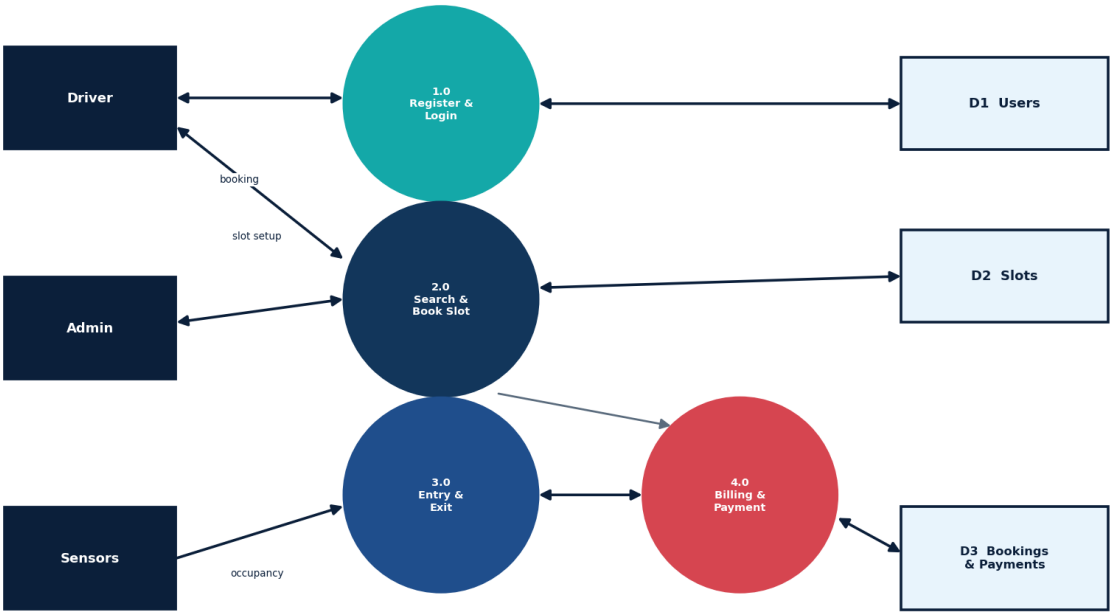


Figure 13.2: Level 1 DFD

Explanation: Login data goes to process 1.0, which checks the Users store (D1). Booking requests go to process 2.0, which reads and updates the Slots store (D2). Sensor readings and gate events go to process 3.0. Process 4.0 calculates the fee and saves bookings and payments in D3.

Chapter 14: Database Design

The Entity-Relationship (ER) diagram shows the main tables and how they are related. A user owns many vehicles and makes many bookings; a slot can be booked many times over time; each booking has one payment.

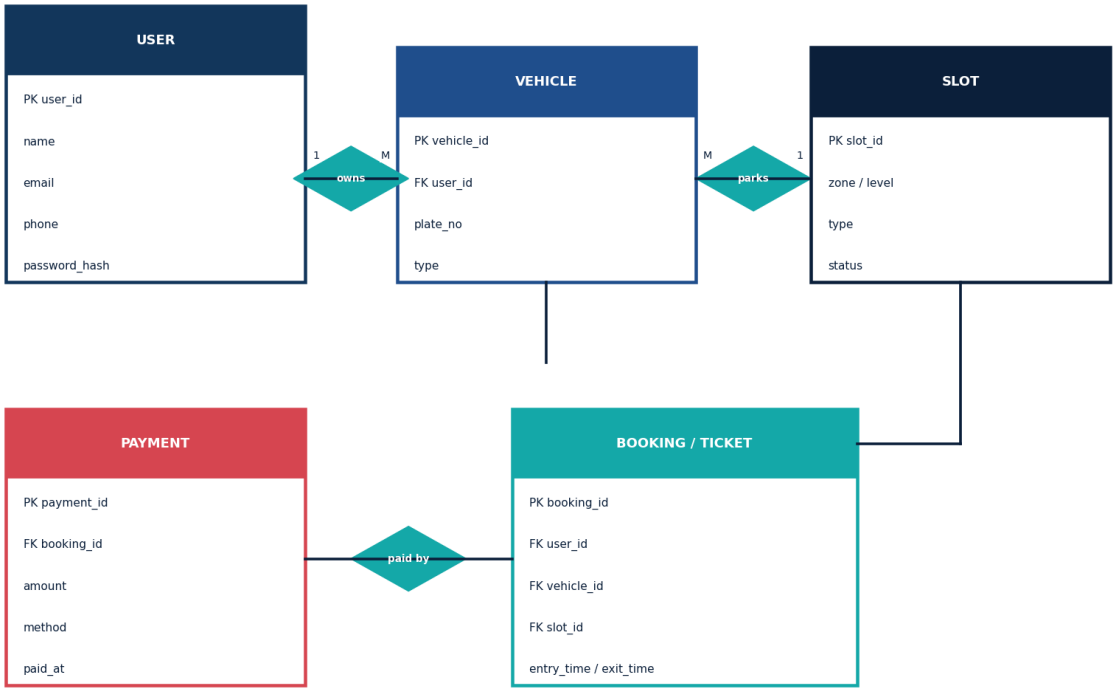


Figure 14.1: Entity-Relationship (ER) diagram

Table 14.1: Table structure

Table	Fields
USER	user_id (PK), name, email, phone, password_hash
VEHICLE	vehicle_id (PK), user_id (FK), plate_no, type
SLOT	slot_id (PK), zone, type, status
BOOKING	booking_id (PK), user_id (FK), vehicle_id (FK), slot_id (FK), entry_time, exit_time
PAYMENT	payment_id (PK), booking_id (FK), amount, method, paid_at

PK means primary key (a unique ID) and FK means foreign key (a link to another table). Passwords are stored only as hashes.

Chapter 15: UML Design Models

Besides the use case diagram, three more UML diagrams explain the design: the class diagram (structure), the sequence diagram (order of messages) and the state diagram (behaviour of a slot).

15.1 Class Diagram

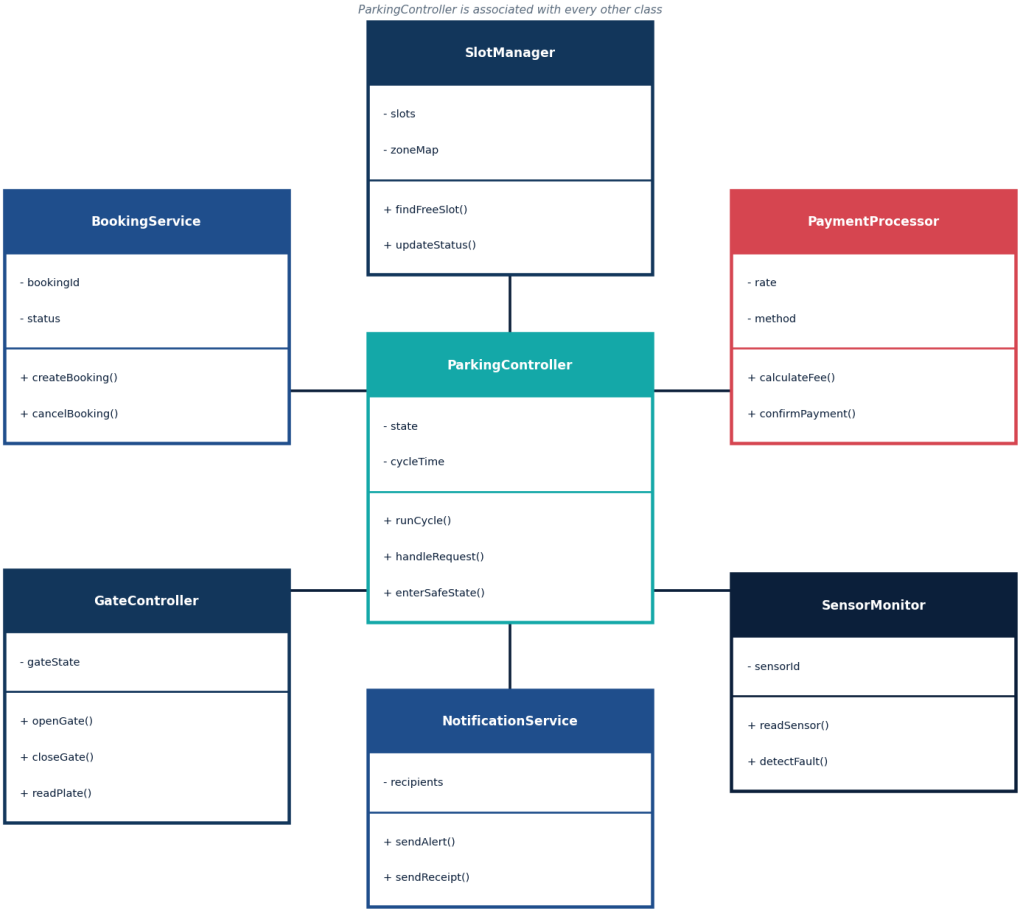


Figure 15.1: UML class diagram of the main classes

15.2 Sequence Diagram

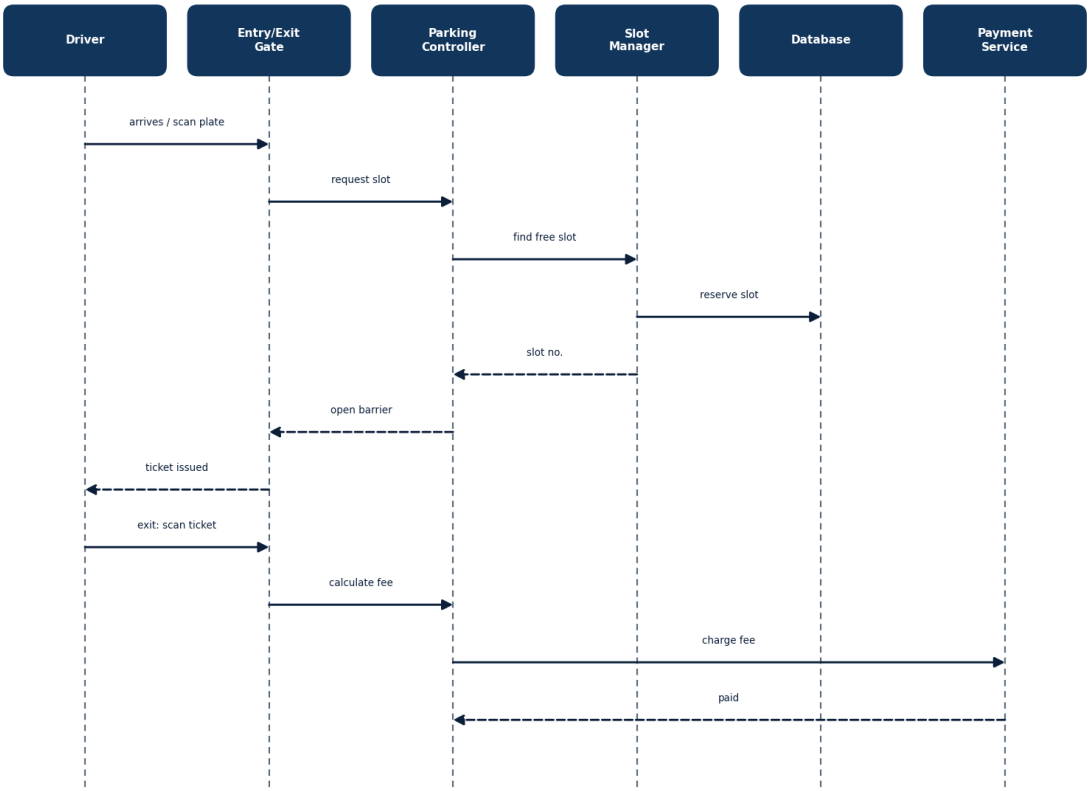


Figure 15.2: UML sequence diagram for a vehicle entry and exit

15.3 State Diagram

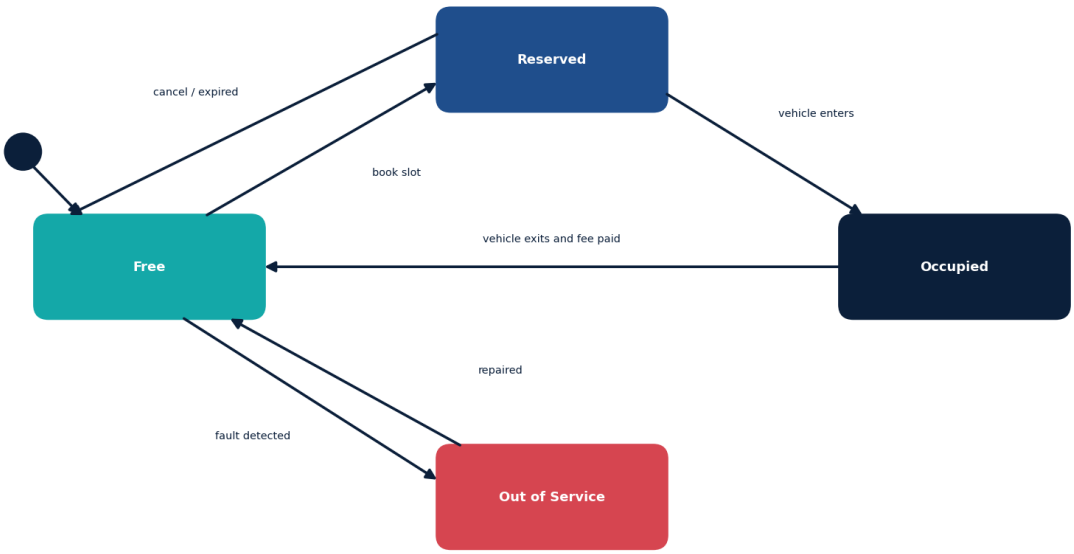


Figure 15.3: UML state diagram of a parking slot

Explanation: A slot starts as Free. A booking makes it Reserved, and when the vehicle enters it becomes Occupied. After exit and payment it returns to Free. A cancelled or expired booking also returns it to Free. A faulty sensor moves it to Out of Service until it is repaired.

Chapter 16: Software Development Life Cycle (SDLC)

SDLC is the step-by-step process of building software.



Figure 16.1: SDLC phases

Table 16.1: SDLC phases and activities

Phase	Activity for this case study
Planning	Define goals, scope, users and feasibility.
Requirements	Collect needs of drivers, attendants and admins.
Design	Prepare architecture, database, flowchart, UML and DFD.
Implementation	Code the services, database and client screens.
Testing	Unit, integration, system and acceptance testing.
Deployment	Install the software and connect sensors and gates.
Maintenance	Fix bugs, update rates and add features.

For a system like this, an iterative approach works well: a first version (login, slot view, entry and exit with billing) is released and tested, and feedback is used to add booking, alerts and reports in later versions. Careful documentation is still needed because the software controls gates and handles payments.

Chapter 17: Implementation Overview

This chapter gives a simple idea of how the design can be coded. It is a teaching sketch only.

17.1 Suggested Technology

- Client: HTML, CSS and JavaScript (web), or a mobile app for drivers and a kiosk screen at the gate.
- Server: Python, Java or Node.js providing an API.
- Device link: sensors and barriers connected through a small gateway using a standard protocol such as MQTT.
- Database: MySQL or PostgreSQL.
- Each module in the architecture becomes a separate service or class.

17.2 Vehicle Entry Process (Pseudocode)

```
FUNCTION vehicle_entry(plate):
    IF NOT SLOT_AVAILABLE():
        SHOW_MESSAGE("Parking full")
        RETURN "Full"
    booking = FIND_BOOKING(plate)
    IF booking IS NOT NULL:
        slot = booking.slot
    ELSE:
        slot = ALLOCATE_SLOT()        // lock the slot first
    ticket = ISSUE_TICKET(plate, slot, NOW)
    SET_SLOT_STATUS(slot, "Reserved")
    OPEN_ENTRY_BARRIER()
    LOG("entry", ticket)
    RETURN "Entered"
```

17.3 Vehicle Exit and Payment (Pseudocode)

```
FUNCTION vehicle_exit(ticket_id):
    ticket = FIND_TICKET(ticket_id)
    IF ticket IS NULL:
        RETURN "Invalid ticket"
    fee = CALCULATE_FEE(ticket.entry_time, NOW)
    IF NOT PAYMENT_SUCCESS(fee):
        SHOW_MESSAGE("Payment failed")
        RETURN "Retry"
    OPEN_EXIT_BARRIER()
    SET_SLOT_STATUS(ticket.slot, "Free")
    LOG("exit", ticket)
    RETURN "Exited"
```

Note: the rate table and the grace time used in CALCULATE_FEE are set by the parking operator; they are not fixed by this project.

17.4 Coding Practices

- Keep functions small and test each one alone.
- Validate every input before using it.
- Lock a slot while it is being allocated, so that two vehicles never get the same slot.
- Never ignore errors; log them.
- Use meaningful names and comments for easy maintenance.

Chapter 18: Security and Risk Management

A parking platform stores accounts, vehicle numbers and payment records, and it controls physical gates, so security and safety are very important. Risk management means identifying, assessing, reducing and monitoring risks.

Table 18.1: Security measures

Area	Measures
Authentication	Login with password; lock or slow down after repeated failures.
Password protection	Store passwords only as strong hashes, never as plain text.
Authorisation	Role-based access: drivers, attendants and admins can do different things.
Secure connection	Use encrypted connections (HTTPS / TLS) for data in transit.
Payment safety	Use a trusted payment provider; do not store card numbers.
Input validation	Clean all inputs to block script injection and database attacks.
Gate safety	Fail-safe manual mode so that gates can be operated if the software fails.
Audit log	Record entries, exits and important admin actions.

Table 18.2: Sample risk table

Risk	Possible Effect	Control
Sensor failure	Wrong slot status	Fault detection and manual check
Server down	Gates cannot be controlled	Backup server and manual mode
Duplicate slot allocation	Two cars for one slot	Lock the slot during allocation
Account takeover	Misuse of user's account	Strong passwords, lockout, secure login
Data leak	Private data exposed	Access control and encryption
Payment error	Wrong charge	Receipts, logs and a refund process
Script injection	Attack through input fields	Input validation and output encoding

Chapter 19: Software Testing

Testing checks that the software does what it should. The test cases below are samples only.

Table 19.1: Sample test cases

ID	Test Case	Input / Condition	Expected Result
TC01	Register user	Valid new details	Account created
TC02	Invalid login	Wrong password	Access denied; error message
TC03	Valid login	Correct ID and password	Access granted
TC04	View slots	Open the slot map	Live Free / Occupied status shown
TC05	Book a free slot	Select a free slot	Slot becomes Reserved
TC06	Book a taken slot	Select an occupied slot	Booking refused
TC07	Vehicle entry	Free slot available	Ticket issued; barrier opens
TC08	Lot full	No free slot at entry	"Parking Full" message
TC09	Fee calculation	Parked for a known time	Correct fee shown
TC10	Payment success	Valid payment	Exit opens; slot becomes Free
TC11	Payment failure	Payment declined	Exit stays closed; retry allowed
TC12	Sensor fault	Sensor stops responding	Slot Out of Service; alert sent
TC13	Add slot	Admin adds a new slot	Slot appears in the slot map

19.1 Levels of Testing

- Unit testing: each module is tested alone.
- Integration testing: modules are tested together.
- System testing: the complete system is tested.
- Acceptance testing: users check that their needs are met.

19.2 Testing Methods

- Black-box testing: test inputs and outputs without looking at the code.
- White-box testing: test internal paths and decisions in the code.
- Boundary value testing: test at limits such as a full lot or zero minutes parked.
- Load testing: check behaviour when many vehicles enter at the same time.

Chapter 20: Ethical and Professional Considerations

Software engineers who build systems that handle personal data and control physical gates have a serious responsibility. The ACM/IEEE Software Engineering Code of Ethics asks engineers to act in the public interest.

- Privacy: protect vehicle numbers and personal data; collect only what is needed.
- Safety: never release a gate controller that is not properly tested, and always keep a safe manual mode.
- Fairness: apply rates and booking rules clearly and equally to all users.
- Honesty: be clear about what data is stored and how it is used.
- Accessibility: reserve and show accessible slots, and design screens that more people can use.
- Accountability: keep logs so that decisions can be reviewed.

This project follows these ideas by using no real user data and by including safety, privacy and audit features in the design.

Chapter 21: Advantages

A smart parking management system offers the following advantages:

- Drivers save time because they can see free slots.
- Fewer manual steps at the gates and shorter queues.
- Automatic fee calculation reduces errors and disputes.
- Digital payment reduces cash handling.
- Slot booking helps drivers plan ahead.
- Better record keeping through the event log.
- Easy reports on occupancy and income for the operator.
- Faster detection of faults since the system checks sensors regularly.
- Less fuel wasted and less traffic caused by searching for a slot.
- Roles and access control keep admin functions safe.
- Modular design makes testing and updates easier.
- Models such as UML, DFD and ER diagrams make the system easy to understand.

- The case study helps students learn the real steps of the SDLC.

Chapter 22: Limitations

This case study and the proposed design also have some limitations:

- The system depends on correct sensors; a hidden defect can cause errors.
- It needs a stable network connection and power backup.
- The complete software and hardware are not built in this case study.
- Real-time systems are complex and costly to build and run.
- Number-plate reading can fail in poor light or with dirty plates.
- Security needs constant attention; weak passwords can cause risks.
- Private data must be protected, or privacy can be harmed.
- Users need training to use the app, kiosks and messages correctly.
- Only a conceptual design and sample test cases are presented.
- Testing cannot prove that every possible fault has been found.
- Software updates must be controlled carefully, because new changes can introduce errors.
- The design does not describe any real commercial product.

Chapter 23: Future Scope

- Number-plate recognition with better accuracy.
- Dynamic pricing based on demand.
- Navigation inside large lots with a mobile app.
- Integration with city-wide parking and traffic data.
- Electric vehicle charging slot management.
- Subscription and season passes for regular users.
- Analytics dashboards for the operator.
- A working prototype built in a classroom using simulated sensors.

Chapter 24: Conclusion

This case study showed how Software Engineering concepts apply to a smart parking management system. Requirements, a layered architecture, a flowchart, UML, DFD and ER models, the SDLC, security measures and test cases together help to build dependable software. Accurate slot status, safe gate control and correct billing are the key design points.

The study also showed why careful testing, security and ethics matter when software controls gates and handles personal data and payments. This is an independent educational project; the design is conceptual and uses no real user data.

The work has improved my understanding of the Software Engineering subject and how its models are used in real problems.

References

[1] I. Sommerville, Software Engineering, Pearson Education.

[2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.

[3] P. Jalote, An Integrated Approach to Software Engineering, Springer.

[4] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson.

[5] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, Pearson.

[6] Object Management Group (OMG), Unified Modeling Language (UML) Specification, omg.org.

[7] ISO/IEC/IEEE 29148, Systems and software engineering: Life cycle processes: Requirements engineering.

[8] ISO/IEC 25010, Systems and software engineering: Systems and software quality models.

[9] ACM/IEEE-CS, Software Engineering Code of Ethics and Professional Practice, acm.org.

[10] OWASP Foundation, OWASP Top 10 Web Application Security Risks, owasp.org.

Appendix A: Requirements Traceability Matrix

Traceability links each requirement to a module and a test case, so nothing is left untested.

Table A.1: Requirement traceability

Requirement	Module(s)	Test Case(s)
FR1 Register and log in	Database Tables, Client Layer	TC01, TC02, TC03
FR2 Show live slot status	Sensor Gateway, Slot Allocation	TC04, TC12
FR3 Book and cancel a slot	Booking Service	TC05, TC06
FR4 Allocate slot, issue ticket	Slot Allocation, Gate Controller	TC07, TC08
FR5 Open gate for valid ticket	Gate Controller	TC07, TC11
FR6 Calculate fee	Billing and Payment	TC09
FR7 Accept payment, receipt	Billing and Payment, Alert Service	TC10, TC11
FR8 Admin management and reports	Database Tables, Client Layer	TC13